

AI-Powered Fraud Detection & Investigation Platform

Functionality Checklist & Assessment Mark Scheme

Module: Fintech / Portfolio Project | Team size: 2 | Type: Full-stack ML + Software Engineering

Collaboration model: *feature-slice split* — each person builds one feature end-to-end (data, model/logic, API, UI), then swaps slices at the midpoint so both get full-stack experience.

1. Functionality Checklist

Tick off each item as it's built. Grouped by feature slice, then by shared/cross-cutting work.

1.0 Kickoff (Do Together, Before Splitting)

- ☐ Agree the API contract (request/response shapes) — see Section 4
- ☐ Agree the shared database schema (transactions, cases, feedback, users)
- ☐ Set up shared GitHub repo, branching convention, and PR workflow

1.1 Slice 1: Detection — Data & Model

- ☐ Source a fraud transaction dataset
- ☐ Ingest transaction data (Transaction ID, Customer ID, Amount, Merchant, Location, Time, Device, Payment method)
- ☐ Data cleaning pipeline (missing values, duplicates, categorical encoding)
- ☐ Address class imbalance (SMOTE / class weighting / undersampling)
- ☐ Feature engineering (spend averages, location/device deviation, transaction velocity)
- ☐ Baseline model (e.g. logistic regression / random forest)
- ☐ Neural network model (TensorFlow) for fraud classification
- ☐ Model evaluation: precision, recall, F1, ROC-AUC
- ☐ Risk categorisation logic: map probability to Low / Medium / High
- ☐ Explainability logic (e.g. SHAP or custom feature-contribution reasons)

1.2 Slice 1: Detection — API & UI

- ☐ Prediction endpoint (transaction in → risk score + category + reasons out)
- ☐ Transaction ingestion endpoint
- ☐ Alerts list view (flagged transactions with risk badge)
- ☐ Search & filter alerts by risk level
- ☐ Explanation panel in the UI showing model reasoning
- ☐ Charts / visualisation of risk trends and alert volume

1.3 Slice 2: Investigation — Data & Backend

- ☐ Design case & feedback database schema
- ☐ Case management API (create from alert, view, update, list)
- ☐ Add notes to a case (endpoint + storage)
- ☐ Mark case as resolved endpoint
- ☐ Analyst feedback endpoint (mark alert as Fraud / Genuine)
- ☐ Authentication (analyst login)
- ☐ Customer transaction history endpoint

1.4 Slice 2: Investigation — UI

- ☐ Case investigation view with notes thread
- ☐ Mark case as resolved (UI action)
- ☐ Customer transaction history view
- ☐ Search across cases and transactions
- ☐ Frontend connected to live backend APIs

1.5 Midpoint Swap

- ☐ Each person walks the other through their code (handover session)
- ☐ Person 1 takes over Investigation; Person 2 takes over Detection
- ☐ Each person extends/polishes the slice they inherited (see tracker Phase 2 tasks)

1.6 Feedback Loop (Cross-Slice)

- ☐ Analysts can label alerts as Fraud or Genuine
- ☐ Feedback stored and linked to the original transaction/case
- ☐ Mechanism (even if manual/documented) for feedback to inform model retraining

1.7 Real-Time Processing (Advanced / Stretch)

- ☐ Event-driven pipeline: transaction → processing → ML model → risk score → alert
- ☐ Kafka or RabbitMQ integration
- ☐ Simulated live transaction stream

1.8 Cloud, DevOps & System Design (Shared)

- ☐ Dockerised services (backend, frontend, database)
- ☐ CI/CD pipeline (GitHub Actions)
- ☐ Deployed to Google Cloud / Azure
- ☐ Monitoring and logging in place
- ☐ System architecture diagram produced
- ☐ Consideration of caching, load handling and fault tolerance (even if only discussed in docs)

1.9 Final Deliverables (what you must hand in)

- ☐ Deployed, working web application
- ☐ GitHub repository with clean documentation (README, setup instructions)
- ☐ Architecture diagram
- ☐ Machine learning model report
- ☐ Demo video
- ☐ Technical blog / article explaining design choices and trade-offs

2. API Contract (Agree Before Splitting)

Lock these shapes down together first — both slices depend on them, and disagreeing about them later is expensive to fix.

Endpoint	Method	Used by	Request (key fields)	Response (key fields)
/transactions	POST	Slice 1	transaction_id, customer_id, amount, merchant, location, timestamp, device_id, payment_method	transaction_id, status: “received”
/predict	POST	Slice 1	transaction_id (or full transaction object)	transaction_id, fraud_probability (0–1), risk (Low/Medium/High), reasons: [string]
/alerts	GET	Slice 1 UI	query params: risk, date_range, search	list of: transaction_id, customer_id, amount, risk, fraud_probability, created_at
/customers/{id}/history	GET	Slice 2	customer_id (path)	list of past transactions for that customer
/cases	POST	Slice 2	transaction_id, opened_by	case_id, status: “open”
/cases	GET	Slice 2 UI	query params: status, search	list of: case_id, transaction_id, status, opened_at, last_note
/cases/{id}	GET	Slice 2 UI	case_id (path)	case_id, transaction, risk info, notes: [], status
/cases/{id}/notes	POST	Slice 2	case_id (path), note text, author	note_id, created_at
/cases/{id}/resolve	PATCH	Slice 2	case_id (path), resolution (fraud/genuine)	case_id, status: “resolved”
/feedback	POST	Slice 2	transaction_id, label (fraud/genuine), analyst_id	feedback_id, status: “recorded”
/auth/login	POST	Both UIs	username, password	access_token, token_type

3. Mark Scheme

Suggested weighting across 100 marks, based on the brief's core requirements and skills sections. Adapt weightings to match your actual module's assessment criteria if one is issued.

Criterion	What's assessed	Marks
Machine Learning Model & Evaluation	Feature engineering, model choice, handling of class imbalance, evaluation metrics (precision/recall/F1/ROC-AUC), quality of risk scoring	25
Backend & API Design	REST API design, endpoint coverage, authentication, error handling, testing	15
Database & Data Management	Schema design, data integrity, appropriate storage of transactions/cases/feedback	10
Frontend Dashboard & UX	Usability of alert/case views, search & filtering, clarity of information for analysts	15
Explainable AI	Quality and clarity of fraud-reasoning explanations surfaced to the user	10
System Architecture & Integration	Coherence of overall design, quality of architecture diagram, end-to-end integration between ML/backend/frontend	10
Deployment & DevOps	Containerisation, CI/CD, cloud deployment, monitoring/logging	5
Documentation & Portfolio Deliverables	README, model report, demo video, technical blog/article	5
Collaboration & Full-Stack Range	Evidence both members worked across data/ML, backend and frontend via the slice swap (commit history, handover notes)	5
Total		100

3.1 Grade Band Descriptors

Band	Descriptor
Distinction (80–100)	Fully working end-to-end system deployed to the cloud. Model evaluated rigorously with clear metric justification. Explanations are genuinely useful, not generic. Clean, well-tested code and thorough documentation. Real-time/advanced pipeline attempted.
Merit (70–79)	Core pipeline works end-to-end locally or deployed. Solid model evaluation. Dashboard covers all core features. Good documentation; minor gaps in testing or polish.
Pass (60–69)	Core features present but with some rough edges — e.g. limited explainability, basic model evaluation, or a dashboard missing one or two features. Deliverables present but not fully polished.
Marginal (50–59)	Individual components (model, API, dashboard) exist but aren't well integrated. Evaluation is superficial (accuracy only). Documentation is thin.
Fail (<50)	Missing core functionality (e.g. no working model, no dashboard, or no integration between components). Deliverables incomplete.

4. What Else to Add (Beyond the Brief)

Things markers and interviewers commonly look for that aren't spelled out explicitly in the brief but will strengthen the submission:

4.1 Documentation & Presentation

- ☐ Clear README with setup/run instructions, screenshots, and architecture overview
- ☐ API documentation (FastAPI auto-generates Swagger/OpenAPI docs — link to it)
- ☐ Commit history that shows iterative, sensible progress (not one giant commit)
- ☐ A short 'known limitations / future work' section in the report

4.2 Data & Model Rigour

- ☐ Train/validation/test split clearly explained, with no data leakage
- ☐ Confusion matrix and threshold justification (why 92% = high risk, not 80%)
- ☐ Discussion of false positive vs false negative trade-offs in a fraud context
- ☐ Baseline vs final model comparison to show improvement

4.3 Engineering Quality

- ☐ Environment variables / secrets not committed to the repo
- ☐ Basic input validation and error handling on API endpoints
- ☐ Consistent code style/linting across both members' code
- ☐ A .env.example or config template for anyone re-running the project

4.4 Collaboration Evidence

- ☐ Clear record of who owned which parts (README or project board)
- ☐ Evidence of integration work, not just two separate halves stitched together
- ☐ Shared testing of the end-to-end flow (transaction in → alert on dashboard)